Information Protection Assessment & Readiness — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Information Protection Assessment & Readiness (Microsoft Purview) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

An honest note . . . 4

Outcome . . . 5

1. Solution Overview — Information Protection Assessment & Readiness (Microsoft Purview) 5

What this solution is . . . 5

The posture-assessment model in one paragraph . . . 5

What "posture" means here (the six dimensions assessed) . . . 5

What is authored exactly vs. deferred . . . 6

The pilot-in-simulation principle . . . 6

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Information Protection Assessment & Readiness (Microsoft Purview)

2.1 Compliance licensing sufficient to *read* the posture (category G1) . . . 8

2.2 Least-privilege read roles for the assessment (category G2) . . . 8

2.3 Security & Compliance PowerShell + Graph reachable (category G3) . . . 8

2.4 In-scope workloads and data sources defined (category G4) . . . 8

2.5 A readiness baseline to score against . . . 9

2.6 A pilot scope for the one validated fix (category G6) . . . 9

2.7 Stakeholders for determinations (category G5) . . . 9

Checklist . . . 9

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10

2. Purview roles (least-privilege) and portal access . . . 10

3. Security & Compliance PowerShell + Graph toolchain . . . 10

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 11

6. A pilot scope and change control . . . 11

3. Implementation Guide — Information Protection Assessment & Readiness (Microsoft Purview) 12

Phase 1 — Assess (read-only discovery) . . . 12

Phase 2 — Design (score and rank) . . . 13

Phase 3 — Pilot one fix in SIMULATION (safe) . . . 14
Phase 4 — Validate and hand off . . . 15
5. Testing Guide — Information Protection Assessment & Readiness (Microsoft Purview) 15
Test 1 — Sessions and read access are healthy . . . 15
Test 2 — Estate discovery is complete . . . 16
Test 3 — Label posture is captured accurately . . . 16
Test 4 — Publishing/coverage gaps are visible . . . 16
Test 5 — DLP coverage and mode are correct . . . 16
Test 6 — Scoring reflects the agreed baseline . . . 16
Test 7 — Gap register and roadmap are actionable . . . 16
Test 8 — The pilot fix validates in SIMULATION (and nothing is enforced) . . . 17
Test 9 — Rollback leaves no residue . . . 17
Test 10 — The assessment is repeatable . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Information Protection Assessment & Readiness (Microsoft Purview) 17
A. Connectivity & access . . . 17
B. Licensing-gated visibility . . . 18
C. Discovery accuracy . . . 18
D. Scoring & the roadmap . . . 19
E. The simulation pilot . . . 19
Escalation checklist . . . 20
7. Best Practices — Information Protection Assessment & Readiness (Microsoft Purview) 20
Assessing posture . . . 20
Scoring and the roadmap . . . 21
Piloting safely . . . 21
Operating and handing off . . . 21
Automation & security . . . 22
8. Deliverables Summary — Information Protection Assessment & Readiness (Microsoft Purview) 22
Documentation . . . 22
PowerShell scripts (Security & Compliance PowerShell / Microsoft Graph) . . . 23
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Information Protection Assessment & Readiness (Microsoft Purview)

Most organizations label, encrypt, and apply DLP **ad hoc** — a few labels somebody created, a DLP policy or two, auto-labeling half-configured — with **no baseline** of what actually protects the data estate and **no prioritized plan** to close the gaps. That is both a compliance exposure (you cannot show what protects regulated data) and a data-risk exposure (sensitive content is under-protected or over-shared). Before you build more in **Microsoft Purview**, you should know exactly where you stand.

This solution delivers an **information-protection posture assessment** end to end: it inventories the sensitivity **labels**, label **publishing**, **encryption/protection** settings, **DLP** policies and rules, **auto-labeling**, and the **sensitive information types** in scope; scores them against a readiness baseline; produces a **gap register** and a **prioritized information-protection roadmap** (quick wins vs. strategic work); validates a top recommended fix by **piloting it in simulation/test** (labeling and DLP support a simulation mode that reports what *would* happen and changes nothing); and hands you a defensible **posture scorecard** — configured, piloted, validated, and operable.

What you get

- A plain-language **Solution Overview** of the posture-assessment method and the

Assess → Design → Configure → Pilot (simulate) → Validate → Expand model.

- A **Requirements Guide** with verify / implement-if-missing for every prerequisite (the category

Purview requirements — licensing, least-privilege roles, in-scope data, stakeholders, pilot — are referenced, not repeated).

- An **Implementation Guide** of precise, ordered **Purview portal + Security & Compliance PowerShell +

Graph **steps: run the discovery, build the gap register, produce the roadmap, pilot one fix in** simulation**, and validate — each artifact named, with pass/fail gates and rollback.

- Five **PowerShell scripts**: posture Discovery, a Scope/Health helper, a simulation-only Pilot

Remediation (`-WhatIf` by default), a Validation check, and a Roadmap/Scorecard Report.

- **Testing** and **Troubleshooting** guides for a confident, defensible assessment.

What this delivers

A **baseline you can defend**: what protects the estate today, where the gaps are, and a ranked plan to close them — with one fix proven in simulation so the roadmap is credible, not theoretical.

The deployment model

Assess → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand** — measure posture, rank the gaps, prove a fix safely in simulation, then enforce and expand under change control as follow-on work.

An honest note

This is an **assessment and readiness** engagement: it measures posture and validates a fix path in **simulation** — it does **not** enforce labels, DLP, or auto-labeling tenant-wide (that is change-controlled follow-on work). Purview capabilities are **licensing-dependent** (many need **Microsoft 365 E5 / E5 Compliance**; some are in preview) and the portal evolves — availability and paths are **cited or flagged "verify your licensing/tenant."** Regulatory, records, and legal determinations (retention schedules, records file plans, legal holds) are **deferred** to the organization's compliance, records, privacy, and legal functions.

Outcome

A measured, evidenced information-protection posture, a prioritized roadmap, and a validated fix path — so the capability is in place, defensible, and ready to expand.

1. Solution Overview — Information Protection Assessment & Readiness (Microsoft Purview)

What this solution is

This solution delivers a **posture assessment and readiness roadmap** for **information protection** in **Microsoft Purview** — the sensitivity **labels**, **encryption/protection**, **content marking**, **label publishing**, **Data Loss Prevention (DLP)**, **auto-labeling**, and the **sensitive information types** that together protect an organization's data. It answers three questions with evidence: **what protects the estate today, where the gaps are, and what to fix first.**

It is deliberately an **assessment**, not a rollout. It **measures** the current posture, **scores** it against a readiness baseline, produces a **gap register** and a **prioritized information-protection roadmap**, and **validates one recommended fix by piloting it in simulation/test** — a mode that reports what a policy *would* do and changes nothing. Enforcing labels/DLP/auto-labeling tenant-wide is **change-controlled follow-on work** and is out of scope by design.

The posture-assessment model in one paragraph

Information protection in Purview is a stack of controls: **labels** classify (and can **encrypt** and **mark**) content; **label policies** publish labels to users/locations; **auto-labeling** applies labels to content at rest or in transit based on **sensitive information types (SITs)** or trainable classifiers; and **DLP** detects and (when enforced) restricts risky handling of that content. A control only protects data if it is **created, published, in the right mode, and scoped to the right workloads**. This assessment walks that stack, records the state of each control, and finds where the chain is broken — an unpublished label, a DLP policy stuck in test, a workload with no coverage, a label that classifies but never encrypts.

What "posture" means here (the six dimensions assessed)

1. **Classification taxonomy** — do sensitivity labels exist, are they enabled, and is the label set coherent (not a sprawl of orphan labels)?
1. **Label publishing & coverage** — are labels **published** (via label policies) to the right

users/groups and locations, or do they sit unused?

1. **Encryption & protection** — do the labels that should encrypt actually apply **encryption /

rights protection**, and are content markings (header/footer/watermark) set where required?

1. **Auto-labeling** — is **auto-labeling** configured, and in what **mode** (simulation vs. enabled),

for content at rest (SharePoint/OneDrive/Exchange) and client-side?

1. **DLP coverage & mode** — which **workloads** (Exchange, SharePoint, OneDrive, Teams, Endpoint,

Defender for Cloud Apps) have DLP, and are policies **enabled** or stuck in **test/simulation**?

1. **Sensitive information types** — are the **SITs** that define "sensitive" for this org present

(built-in + any custom), and do labels/DLP actually reference them?

Each dimension yields findings; findings become **gaps**; gaps are **ranked** into the roadmap.

What is authored exactly vs. deferred

- **Exact detail is given** for **Security & Compliance PowerShell** (`Connect-IPPSSession`:

`Get-Label`, `Get-LabelPolicy`, `Get-DlpCompliancePolicy`, `Get-DlpComplianceRule`, `Get-AutoSensitivityLabelPolicy`, `Get-DlpSensitiveInformationType`) and the **Microsoft Purview portal** (`purview.microsoft.com`) navigation for **Information Protection**, **Data Loss Prevention**, and **Content/Activity Explorer**.

- **Licensing-gated capability** (auto-labeling, advanced DLP, Data Security Posture Management / DSPM)

is **flagged**: where a feature is not readable the assessment records it as **"not available — verify your licensing/tenant"** rather than asserting it exists.

- **Content Explorer / Activity Explorer / DSPM** are **portal surfaces** with their own role

requirements; the assessment cites where to read coverage there and does **not** invent an API that scrapes their contents.

- **Regulatory, records, and legal determinations** (what *must* be labeled/retained, records file

plans, legal holds, regulatory applicability) are **deferred** to compliance/records/privacy/legal — this solution provides the technical posture and evidence, **not** the determination.

The pilot-in-simulation principle

Every high-impact Purview control supports a **safe pre-enforcement mode**, and this assessment uses it:

- **Auto-labeling** runs in **simulation** (`Mode = TestWithoutNotifications`) — it evaluates content and

reports what it **would** label, and applies **nothing**.

- **DLP** policies can be created in **test/simulation** (test with or without policy tips) before

`Enable` — they detect and report without blocking.

- **Label publishing** can be piloted to a **small pilot group** before tenant-wide publish.

The single fix this engagement validates is proven in **simulation** — never enforced — so the roadmap is credible without touching production behavior.

Benefits

- **Know before you build** — a measured baseline of what protects the estate, not guesswork.
- **Prioritized, not overwhelming** — gaps ranked into quick wins vs. strategic work.
- **Defensible** — an evidenced posture scorecard you can show compliance and leadership.
- **Safe** — the validated fix is proven in **simulation**; nothing is enforced.
- **Repeatable** — re-run the discovery/scorecard to measure improvement over time.

Limitations (be honest with buyers)

- **Assessment, not enforcement.** This measures posture and validates a fix in **simulation**; it does **not** roll out labels/DLP/auto-labeling. Enforcement is change-controlled follow-on work.
- **Licensing-dependent visibility.** Advanced DLP, auto-labeling, and DSPM require **E5 / E5 Compliance**; **where unlicensed, those dimensions are recorded as** not available**, not assumed.
- **Portal-only surfaces.** Content Explorer / Activity Explorer / DSPM coverage is read in the portal under their own roles — the scripts inventory policy *configuration*, not per-item content.
- **No determinations.** Whether specific data *must* be protected/retained is a compliance/records/legal call — deferred by design.

The standard deployment model

> ***Assess the posture → design the target state + roadmap → configure the assessment artifacts → pilot one recommended fix in SIMULATION → validate it would work → expand (enforce) as follow-on.*** *Pattern:* ***Assess → Design → Configure → Pilot (simulate) → Validate → Expand****, with explicit pass/fail gates and a documented rollback for anything created.*

2. Requirements Guide — Information Protection Assessment & Readiness (Microsoft Purview)

Verify **every** requirement before running the assessment. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide Purview prerequisites (a tenant with the required **compliance licensing**, least-privilege **Purview roles**, the **Security & Compliance PowerShell + Graph** toolchain, **in-scope data sources**, **compliance/records/privacy/legal stakeholders**, and a **pilot scope + change control**) are in `global-requirements-purview.md` — confirm those **first**; this guide adds the assessment-specific items and does **not** duplicate the six.

> ***Artifact names used throughout (use verbatim):*** *posture discovery output* `Information Protection - Estate - Discovery` *(CSV); gap register* `Information Protection - Gaps - Register` *(CSV); roadmap/scorecard* `Information Protection - Roadmap - Scorecard`*;*

simulation pilot policy (auto-labeling) `Information Protection - Pilot - Simulation`*; pilot label policy (optional)* `Information Protection - Label Policy - Pilot`*; pilot scope group* `Information Protection - Pilot Users`*.*

2.1 Compliance licensing sufficient to *read* the posture (category G1)

Description: A Microsoft 365 tenant where the information-protection controls you want to assess are **visible**. Manual sensitivity labeling + basic DLP are **E3**-class; **auto-labeling**, **advanced DLP**, and **DSPM** need **E5 / E5 Compliance** (some DSPM/AI features are in **preview**). **Why required:** The assessment can only report on controls the tenant can surface; an unlicensed capability reads as **"not available,"** which is itself a valid finding but must not be mistaken for a gap. **How to verify:** M365 admin center → subscriptions/add-ons; confirm **Information Protection** and **DLP** are visible in the Purview portal. Run `IPASSESS-ScopeHealth.ps1` — it reports which capabilities are readable. **How to implement if missing:** Assess what you can under current licensing; flag licensing-gated dimensions as **not available — verify your licensing/tenant**; recommend licensing in the roadmap if the capability is required.

2.2 Least-privilege read roles for the assessment (category G2)

Description: Read access via the least-privilege Purview roles: **Information Protection Reader** (or **Information Protection Analyst**) for labels/auto-labeling, **Compliance Data Administrator** or **Global Reader** for DLP visibility, and — to read label **coverage** in **Content Explorer** — the **Content Explorer List Viewer / Content Viewer** roles. No enforcement rights are needed to assess. **Why required:** Purview uses fine-grained role groups; the discovery cmdlets and portal surfaces return data only with the matching read role. **How to verify:** Purview portal → **Roles & scopes** → **Permissions**; confirm membership. Run `IPASSESS-ScopeHealth.ps1` — it fails clearly if the S&C session lacks read access. **How to implement if missing:** Assign the least-privilege **reader** role groups under change control. The **Pilot Remediation** step (2.6) additionally needs an **Information Protection Admin** to *create* a simulation policy — grant that only for the pilot.

2.3 Security & Compliance PowerShell + Graph reachable (category G3)

Description: A workstation with **ExchangeOnlineManagement** (for `Connect-IPPSSession`) and **Microsoft Graph PowerShell**, PowerShell 5.1+ or 7+, modern-auth / MFA admin. **Why required:** Every discovery, validation, and reporting script runs through Security & Compliance PowerShell (labels/DLP/auto-labeling) and, optionally, Graph (group scope). **How to verify:** `Connect-IPPSSession` succeeds and `Get-Label` returns without error; `Connect-MgGraph` succeeds. `IPASSESS-ScopeHealth.ps1` confirms both. **How to implement if missing:** `Install-Module ExchangeOnlineManagement` and `Install-Module Microsoft.Graph`; confirm modern-auth sign-in before running steps.

2.4 In-scope workloads and data sources defined (category G4)

Description: The **workloads** the assessment covers — **Exchange, SharePoint, OneDrive, Teams, Endpoint (Windows/macOS), Defender for Cloud Apps** — and the **data-classification / sensitivity intent** that "good posture" is measured against. **Why required:** Posture is measured **per workload**; "coverage" is meaningless without knowing which workloads are in scope, and gaps are ranked against the intended sensitivity scheme. **How to verify:** Confirm the in-scope workloads and the classification intent with stakeholders and record them in the assessment scope. **How to implement if missing:** Agree

the workload scope and the sensitivity intent **before** scoring — otherwise every workload with no coverage reads as a gap regardless of intent.

2.5 A readiness baseline to score against

Description: The **baseline** the current posture is compared to — the six dimensions in the Overview (taxonomy, publishing/coverage, encryption/protection, auto-labeling, DLP coverage/mode, SITs), each with a target the organization considers "ready." The engagement ships a default baseline; refine it with the stakeholders. **Why required:** A gap is the distance between **current** and **target**; without an agreed target the scorecard is subjective. **How to verify:** The baseline dimensions and their targets are written down and agreed by compliance/security stakeholders. **How to implement if missing:** Use the default baseline shipped in `IPASSESS-Validation.ps1` / `IPASSESS-Report.ps1` as a starting point and tune the targets with stakeholders before finalizing.

2.6 A pilot scope for the one validated fix (category G6)

Description: A **small pilot group/scope** (`Information Protection - Pilot Users`) and a **change-control** window so the single recommended fix can be validated in **simulation** without touching production. Auto-labeling simulation applies **nothing**; DLP test mode blocks **nothing**; label pilots publish to the **pilot group only**. **Why required:** Even a simulation policy is a created object; the model is **pilot-in-simulation first**, and creating it should follow change control. **How to verify:** A named pilot group and an approval window exist; the operator has (temporary) **Information Protection Admin** for the pilot only. **How to implement if missing:** Define the pilot group and agree change control before the Pilot Remediation step; keep the rest of the assessment **read-only**.

2.7 Stakeholders for determinations (category G5)

Description: Named **compliance, records, privacy, and legal** stakeholders who own the **determinations** the assessment defers — what data *must* be labeled/encrypted/retained, regulatory applicability, and whether a recommended fix should ultimately be enforced. **Why required:** The assessment provides the technical posture and a ranked roadmap; **whether** to act on a gap (and how far) is a governance decision, not a technical default. **How to verify:** The owners are identified and will review the gap register and roadmap. **How to implement if missing:** Identify the owners before delivering the roadmap so recommendations map to an accountable decision-maker.

Checklist

- Compliance licensing sufficient to read the posture; gated features flagged (2.1 / G1)
- Least-privilege **reader** roles assigned; IP Admin only for the pilot (2.2 / G2)
- `Connect-IPPSSession` + `Connect-MgGraph` succeed on the workstation (2.3 / G3)
- In-scope workloads + classification intent defined (2.4 / G4)
- Readiness baseline (six dimensions + targets) agreed (2.5)
- Pilot group `Information Protection - Pilot Users` + change window for the one fix (2.6 / G6)

• Compliance/records/privacy/legal stakeholders named for determinations (2.7 / G5)

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Information Protection Assessment & Readiness (Microsoft Purview)

Follow these steps **in order**. The **Purview portal** and **Security & Compliance PowerShell** / **Graph** steps are given cmdlet-/click-exact. This is an **assessment**: Steps 1–5 are **read-only** discovery, scoring, and roadmap; Step 6 pilots **one** recommended fix in **simulation** (creates a policy that changes nothing); Step 7 validates; Step 8 hands off to change-controlled follow-on. No labels, DLP, or auto-labeling are **enforced** by this engagement.

Consoles / endpoints used:

- **Microsoft Purview portal** — `https://purview.microsoft.com` (Information Protection, DLP, Content/Activity Explorer; DSPM where licensed)
- **Security & Compliance PowerShell** — `Connect-IPPSSession` (labels, DLP, auto-labeling, SITs)
- **Microsoft Graph** — `Connect-MgGraph` (pilot group membership, optional)

> ***Model:** Measure posture (read-only) → rank gaps → produce the roadmap → pilot **one** fix in **simulation** → validate → hand off enforcement. **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand.***
>
> ***Artifact names (use verbatim):** discovery `Information Protection - Estate - Discovery`; gap register `Information Protection - Gaps - Register`; roadmap/scorecard `Information Protection - Roadmap - Scorecard`; simulation pilot policy `Information Protection - Pilot - Simulation`; optional pilot label policy `Information Protection - Label Policy - Pilot`; pilot scope group `Information Protection - Pilot Users`.*
>
> ***Risk + rollback (read first):** Steps 1–5 and 7 are **read-only** — they create only CSV/report files locally and change nothing in the tenant. **Step 6** creates an **auto-labeling policy in SIMULATION** (`Mode = TestWithoutNotifications`) — it evaluates content and reports what it *would* label and **applies no label to anything**; it is still a created object. **Rollback for Step 6:** delete the simulation policy (`Remove-AutoSensitivityLabelPolicy "Information Protection - Pilot - Simulation"`) — because it labeled nothing, there is nothing to revert. The optional pilot label policy publishes to `Information Protection - Pilot Users` **only**; unpublish it to roll back. **Never enable or enforce** any policy in this engagement.*

Phase 1 — Assess (read-only discovery)

Step 1 — Confirm scope and connectivity

1. Confirm the in-scope workloads and classification intent (Requirement 2.4) and the readiness baseline

(2.5) are agreed and written down.

1. Connect: `Connect-IPPSSession` (and `Connect-MgGraph -Scopes "Group.Read.All"` if you will pilot to a

group in Step 6). Run `IPASSESS-ScopeHealth.ps1` — it confirms the S&C + Graph sessions, reports which capabilities (labels, DLP, auto-labeling, SITs) are **readable**, and flags any that are **not available — verify your licensing/tenant** (Requirement 2.1). Gated features are recorded, not assumed.

Step 2 — Inventory the information-protection estate

1. Run `IPASSESS-Discovery.ps1` (read-only). It inventories, via Security & Compliance PowerShell:

• **Sensitivity labels** (`Get-Label`) — name, priority, enabled, and the **protection** settings that

matter to posture: `EncryptionEnabled`, `EncryptionProtectionType`, content markings (`ApplyContentMarkingHeaderEnabled` / `...FooterEnabled` / `ApplyWaterMarkingEnabled`), and `ContentType` (the label's scope: File/Email, Site/Group, etc.).

• **Label policies** (`Get-LabelPolicy`) — which labels are **published**, and to whom (the coverage

that turns a created label into a used one).

• **DLP** (`Get-DlpCompliancePolicy` / `Get-DlpComplianceRule`) — policy **Mode** (Enable / Test with

or without notifications / Disable) and **Workload** coverage, plus the rules' conditions/actions.

• **Auto-labeling** (`Get-AutoSensitivityLabelPolicy`) — each policy's **Mode** (simulation vs. enabled)

and the label it applies.

• **Sensitive information types** (`Get-DlpSensitiveInformationType`, with `-IncludeSensitiveTypes`) —

built-in + custom SITs that define "sensitive" for the tenant.

1. Output: `Information Protection - Estate - Discovery` (CSV). This is the **"what protects the estate

today"** baseline.

1. In the portal, complement the script with **coverage** you cannot fully script: Purview →

Information Protection → **Content explorer** (label distribution across SharePoint/OneDrive/Exchange) and **Activity explorer** (labeling activity). Note counts for the scorecard. If **DSPM** is licensed, note its recommendations too (**verify your licensing/tenant**).

Phase 2 — Design (score and rank)

Step 3 — Score the posture against the baseline

1. Run `IPASSESS-Validation.ps1` (read-only) against the discovery CSV. It scores each of the six

dimensions (taxonomy, publishing/coverage, encryption/protection, auto-labeling, DLP coverage/mode, SITs) **PASS / WARN / FAIL** against the baseline (Requirement 2.5) — e.g. a label that exists but is **not published** is a WARN; a DLP policy **stuck in Test** across all in-scope workloads is a WARN; an in-scope workload with **no DLP at all** is a FAIL; a "Confidential"-tier label with `EncryptionEnabled = False` is flagged for review.

1. Review each PASS/WARN/FAIL with the stakeholders and confirm the target where the default baseline

doesn't fit the organization.

Step 4 — Build the gap register

1. Run `IPASSESS-Report.ps1-Section GapRegister` (read-only). It turns the scored findings into

`Information Protection - Gaps - Register` (CSV): one row per gap with **dimension, finding, severity, recommended action, effort, and owner-function** (compliance/records/security).

1. Confirm each gap with the accountable stakeholder (Requirement 2.7). **Determinations** (whether a gap

must be closed for a regulation) are recorded against the owner function — **not** decided here.

Step 5 — Produce the prioritized roadmap and scorecard

1. Run `IPASSESS-Report.ps1 -Section Roadmap` (read-only). It ranks the gap register into

`Information Protection - Roadmap - Scorecard`: **quick wins** (publish an existing label, move a DLP policy from Test toward pilot, enable a missing content marking) vs. **strategic** (stand up auto-labeling, extend DLP to a new workload, build custom SITs), each with severity and effort, plus a one-page **posture scorecard** (score per dimension).

1. This is the primary deliverable. Deliver it to the stakeholders as the decision artifact.

Phase 3 — Pilot one fix in SIMULATION (safe)

Step 6 — Validate the top recommended fix in simulation (creates nothing enforceable)

Pick the **highest-value, lowest-risk** gap that can be proven in simulation — typically **auto-labeling a well-defined SIT to an existing label**.

1. Preview first (no change): `.\IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim -SensitivityLabel

"Confidential" -WhatIf`. It prints exactly what it would create.

1. Create the **simulation** policy: `.\IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim

-SensitivityLabel "Confidential" -Commit`. This runs New-AutoSensitivityLabelPolicy -Name "Information Protection - Pilot - Simulation" -Mode "TestWithoutNotifications" ... — **`simulation`**: it evaluates content and reports what it **`would`** label and **`applies no label`**. Then add a matching rule with your SIT conditions (`New-AutoSensitivityLabelRule`) and review results in the portal.

1. **Alternative low-risk pilot:** publish an **existing unpublished** label to `Information Protection -

Pilot Users **only** (optional pilot label policy Information Protection - Label Policy - Pilot`) to prove the publish path — the pilot group sees the label; no one else is affected.

1. **DLP** gaps are validated by moving a policy toward **Test with policy tips** in the portal (detects

and reports, blocks nothing) — this engagement does **not** Enable DLP.

> *Whatever you pilot, it is **simulation/test/pilot-group only**. Do **not** Enable/enforce. The point is to prove the **fix path**, so the roadmap is credible — not to deploy it.*

Phase 4 — Validate and hand off

Step 7 — Validate the pilot and finalize the deliverable (pilot gate)

1. Review the **simulation results** in the portal (Information Protection → Auto-labeling → the policy →

Simulation): it should show the content it **would** have labeled, confirming the fix path is sound. Re-run IPASSESS-Validation.ps1 to confirm the pilot object exists in the expected (simulation) mode and nothing was enforced. Complete the **Testing Guide** pilot gate.

1. Fold the validated fix into the roadmap as a **proven quick win** and finalize `Information Protection

• Roadmap - Scorecard`.

Step 8 — Hand off enforcement (follow-on, out of scope)

1. Present the scorecard, gap register, and roadmap to the compliance/security stakeholders.
2. **Roll back the pilot** if you do not intend to proceed immediately: `Remove-AutoSensitivityLabelPolicy

"Information Protection - Pilot - Simulation"` (it labeled nothing) and/or unpublish the pilot label policy. Enforcing/expanding any recommendation is a **separate, change-controlled engagement**, not this assessment.

Order of operations

```
confirm scope + connect (ScopeHealth) → discover estate (Discovery, read-only)
→ score vs. baseline (Validation) → build gap register (Report:GapRegister)
→ produce roadmap + scorecard (Report:Roadmap) → pilot ONE fix in SIMULATION
(PilotRemediation -WhatIf → -Commit) → validate simulation results → hand
off enforcement as follow-on (roll back the pilot)
```

5. Testing Guide — Information Protection Assessment & Readiness (Microsoft Purview)

Run after the discovery, scoring, and roadmap are produced (Implementation Steps 2–5) and after the one simulation pilot (Step 6). Back each test with IPASSESS-ScopeHealth.ps1 (connectivity/licensing), IPASSESS-Discovery.ps1 (estate), IPASSESS-Validation.ps1 (scoring), and IPASSESS-Report.ps1 (gap register / roadmap). Everything except Test 8 is **read-only**.

Test 1 — Sessions and read access are healthy

Do: Run `IPASSESS-ScopeHealth.ps1`. **Expected:** `Connect-IPPSSession` and (if used) `Connect-MgGraph` are connected; `Get-Label` returns; the report lists which capabilities are **readable** and flags any as **not available — verify your licensing/tenant** (a valid finding, not a failure).

Test 2 — Estate discovery is complete

Do: Run `IPASSESS-Discovery.ps1` (add `-IncludeSensitiveTypes`). **Expected:** `Information Protection - Estate - Discovery` (CSV) lists labels (with encryption/marking settings), label policies, DLP policies + rules (with Mode/Workload), auto-labeling policies (with Mode), and — with the switch — sensitive information types. Nothing was changed in the tenant.

Test 3 — Label posture is captured accurately

Do: Cross-check three labels from the CSV against Purview → **Information Protection** → **Labels**. **Expected:** For each, `Enabled`, `EncryptionEnabled`, `EncryptionProtectionType`, and content-marking flags match the portal; the label's `ContentType` (scope) is recorded. A label that classifies but does **not** encrypt where it should is visible for review.

Test 4 — Publishing/coverage gaps are visible

Do: Compare `Get-Label` count vs. the labels named across `Get-LabelPolicy` in the CSV; check Content explorer for distribution. **Expected:** Any **created-but-unpublished** label is identifiable; the scorecard's publishing/coverage dimension reflects it. Unpublished labels are the classic "we have labels but nobody uses them" gap.

Test 5 — DLP coverage and mode are correct

Do: In the CSV, review each DLP policy's **Mode** and **Workload**; compare to the in-scope workloads (Requirement 2.4). **Expected:** Policies stuck in **Test** and in-scope workloads with **no DLP** are both surfaced; `IPASSESS-Validation.ps1` marks a no-coverage in-scope workload **FAIL** and an all-test policy set **WARN**.

Test 6 — Scoring reflects the agreed baseline

Do: Run `IPASSESS-Validation.ps1`; review the PASS/WARN/FAIL per dimension. **Expected:** Each of the six dimensions scores against the baseline targets (Requirement 2.5); changing a target and re-running changes the score predictably. No dimension is silently skipped — a licensing-gated one reports **not available**, not PASS.

Test 7 — Gap register and roadmap are actionable

Do: Run `IPASSESS-Report.ps1 -Section GapRegister` then `-Section Roadmap`. **Expected:** `Information Protection - Gaps - Register` has one row per gap with severity, recommended action, effort, and owner-function; `Information Protection - Roadmap - Scorecard` ranks them into quick wins vs. strategic with a per-dimension scorecard. Every gap maps to an accountable function (compliance/records/security) — determinations are attributed, not decided.

Test 8 — The pilot fix validates in SIMULATION (and nothing is enforced)

Do: Run `IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim -SensitivityLabel "Confidential" -WhatIf` (preview), then `-Commit`; review the policy in Purview → **Information Protection** → **Auto-labeling** → **Information Protection - Pilot - Simulation** → **Simulation**. **Expected:** With `-WhatIf`, nothing is created. With `-Commit`, the policy exists in **simulation** (`Mode = TestWithoutNotifications`) and, after its rule runs, shows content it **would** label — while applying **no label to anything**. No DLP is enabled. `IPASSESS-Validation.ps1` confirms the object is in simulation mode.

Test 9 — Rollback leaves no residue

Do: `Remove-AutoSensitivityLabelPolicy "Information Protection - Pilot - Simulation"` (and unpublish the pilot label policy if used); re-run `IPASSESS-Discovery.ps1`. **Expected:** The pilot object is gone; because it labeled nothing, there is nothing to revert; the estate matches the pre-pilot baseline.

Test 10 — The assessment is repeatable

Do: Re-run `IPASSESS-Discovery.ps1` + `IPASSESS-Validation.ps1` + `IPASSESS-Report.ps1`. **Expected:** The same posture reproduces; after a real (follow-on) fix the relevant dimension score improves — the scorecard is a repeatable measure of progress.

Pass/fail gate

Gate	Requirement
Discovery pass	Tests 1–3 pass (sessions healthy; estate + label posture captured accurately)
Assessment pass	Tests 4–7 pass (coverage/DLP gaps visible; scoring matches baseline; register + roadmap actionable)
Pilot pass	Tests 8–10 pass (fix validates in simulation; nothing enforced; clean rollback; repeatable)

> *Do not present the roadmap as final until the Assessment gate passes, and do not proceed to any enforcement (follow-on) until the Pilot gate proves the fix path in **simulation**.*

6. Troubleshooting Guide — Information Protection Assessment & Readiness (Microsoft Purview)

Symptom → likely root cause → fix. Use `IPASSESS-ScopeHealth.ps1` (sessions/readability), `IPASSESS-Discovery.ps1` (estate), `IPASSESS-Validation.ps1` (scoring), and `IPASSESS-Report.ps1` (gap register/roadmap) to confirm before and after each fix. Where a capability or portal path is **licensing-gated or evolving, verify against current Microsoft guidance / your tenant** rather than assuming.

A. Connectivity & access

`Get-Label` / `Get-DlpCompliancePolicy` errors with "not recognized" or an auth prompt

• **Not connected to Security & Compliance PowerShell.** These cmdlets come from the S&C session, not

Exchange Online or Graph. **Fix:** `Connect-IPPSSession` with a modern-auth/MFA admin; re-run `IPASSESS-ScopeHealth.ps1` — it confirms the session before you run Discovery.

• **ExchangeOnlineManagement module missing/old.** `Connect-IPPSSession` ships in that module. **Fix:**

`Install-Module ExchangeOnlineManagement` (or `Update-Module`), then reconnect.

The scripts return far fewer objects than the portal shows

• **Insufficient read role.** Labels need **Information Protection Reader/Analyst**; DLP visibility needs

Compliance Data Administrator / Global Reader. Without them a cmdlet may return an empty set rather than error. **Fix:** assign the least-privilege **reader** role groups (Requirement 2.2) and re-run.

• **Scoped administration.** An administrative unit / scoped role can hide objects. **Fix:** confirm the

operator's scope in **Roles & scopes**; assess with a suitably scoped reader.

B. Licensing-gated visibility

Auto-labeling reads as "not available" / `Get-AutoSensitivityLabelPolicy` errors

• **Capability is E5-gated.** Service-side auto-labeling requires **E5 / E5 Compliance**. **Fix:** this is

a **valid finding**, not a script bug — `IPASSESS-ScopeHealth.ps1` and `IPASSESS-Discovery.ps1` record it as **not available — verify your licensing/tenant**. Put licensing in the roadmap only if the capability is actually required; do **not** score it as a configuration gap.

DSPM / advanced DLP options aren't in the portal

• **Not licensed or in preview.** DSPM and some advanced DLP features are **E5**-class and partly in

preview. **Fix:** confirm the subscription; treat absent features as **not available — verify your licensing/tenant**, and don't assert them in the findings.

C. Discovery accuracy

A label's encryption/marking settings look wrong in the CSV

• **Reading a summary vs. the full label object.** Some protection settings live in advanced/label

properties. **Fix:** cross-check with `Get-Label -Identity "<name>" | Format-List Encryption*, ApplyContentMarking*, ApplyWaterMarking*, ContentType` and Purview → **Information Protection** → **Labels**; the Discovery CSV mirrors these fields (Testing Test 3).

• **Label scope (`ContentType`) confusion.** A label scoped to **Sites/Groups** won't behave like a

File/Email label. **Fix:** read the `Scope=` field in the CSV; a container label with no file/email scope is expected, not a defect.

Content explorer shows coverage the scripts don't

- **By design.** The scripts inventory policy **configuration**; per-item label distribution lives in

Content explorer (its own **Content Viewer/List Viewer** roles). **Fix:** read coverage in the portal and record the counts for the scorecard — the scripts don't scrape per-item content.

D. Scoring & the roadmap

Every workload scores FAIL for DLP even though DLP exists

- **In-scope workload list doesn't match reality.** Validation compares DLP coverage to the workloads you

passed. **Fix:** pass the correct `-InScopeWorkloads` (Requirement 2.4); a workload out of scope shouldn't be scored. Re-run `IPASSESS-Validation.ps1`.

- **Policies genuinely stuck in Test.** All-Test scores **WARN** ("nothing enforced"). **Fix:** that is a

real finding — it becomes a quick-win roadmap item (move toward pilot/enforce as follow-on).

The scorecard feels subjective / stakeholders disagree

- **Baseline targets not agreed.** A gap is the distance to the **target**, and the default baseline may

not fit the org. **Fix:** tune the targets (`-InScopeWorkloads`, `-HighTierLabelPattern`, and the baseline mapping) with compliance/security (Requirement 2.5), then re-run so the score reflects the agreed baseline.

A gap has no clear owner

- **Owner-function not attributed.** Every gap row carries an **OwnerFunction**

(compliance/records/security). **Fix:** confirm the named stakeholder (Requirement 2.7); the assessment **attributes** the determination, it does not make it.

E. The simulation pilot

`IPASSESS-PilotRemediation.ps1` *didn't create anything*

- **Ran with `-WhatIf` (the default).** The script previews unless you add `-Commit`. **Fix:** confirm

the previewed action, then re-run with `-Commit`.

- **A policy of the same name exists.** The script refuses to collide with an existing

`Information Protection - Pilot - Simulation`. **Fix:** review/remove the existing one (`Remove-AutoSensitivityLabelPolicy "Information Protection - Pilot - Simulation"`) and re-run.

The simulation policy exists but shows no would-be matches

- **No rule yet, or SIT conditions too narrow.** The policy needs a matching **rule**

(`New-AutoSensitivityLabelRule`) with your sensitive-info-type conditions, and simulation takes time to evaluate. **Fix:** add the rule, wait for the simulation to run, then review results in the portal (Information Protection → Auto-labeling → the policy → **Simulation**).

I'm worried the pilot enforced something

• **Simulation enforces nothing by design.** `Mode = TestWithoutNotifications` applies **no label** to any

content. **Fix:** run `IPASSESS-Validation.ps1 -CheckPilot` — it confirms the object is in a simulation mode; if `Mode` is **not** a Test mode, investigate (this engagement must not enforce). To be safe, remove the pilot: `Remove-AutoSensitivityLabelPolicy "Information Protection - Pilot - Simulation"`.

Rollback — did anything change?

• **Read-only steps changed nothing; the simulation labeled nothing. Fix:** delete the simulation

policy (and unpublish `Information Protection - Label Policy - Pilot` if you piloted a publish); re-run `IPASSESS-Discovery.ps1` — the estate should match the pre-pilot baseline (Testing Test 9).

Escalation checklist

1. `IPASSESS-ScopeHealth.ps1` — S&C (and Graph) connected; which capabilities are readable vs. not

available (licensing)?

1. `IPASSESS-Discovery.ps1` — labels (with encryption/marking/scope), label policies, DLP (Mode/Workload),

auto-labeling (Mode), SITs all captured?

1. `IPASSESS-Validation.ps1` — six dimensions scored against the **agreed** baseline; `-InScopeWorkloads`

correct?

1. `IPASSESS-Report.ps1` — gap register + roadmap generated; each gap has an owner-function?
2. `IPASSESS-PilotRemediation.ps1 -CheckPilot` (via Validation) — pilot object in **simulation**, nothing

enforced?

1. Still stuck → capture the symptom and **verify current Microsoft guidance / your tenant** for the

licensing-gated or evolving capability (auto-labeling, DSPM, DLP simulation) before finalizing the finding.

7. Best Practices — Information Protection Assessment & Readiness (Microsoft Purview)

Field-proven recommendations for a credible, defensible posture assessment that leads to a roadmap people actually execute.

Assessing posture

• **Measure before you build.** Resist the urge to "just create a few more labels." The value here is the

baseline — what protects the estate today — and you can only improve what you have measured.

- **Read-only until the one pilot.** Steps 1–5 change nothing; keep it that way. Run every discovery, scoring, and reporting script with **reader** roles, and grant **Information Protection Admin** only for the single simulation pilot, then remove it.
- **Distinguish "gap" from "unlicensed."** An E5-only capability that is dark on an E3 tenant is **not a configuration gap **— it is a licensing decision. Record it as** not available — verify your licensing/tenant** and put licensing in the roadmap only if the capability is actually required.
- **Coverage is the hidden gap.** Created-but-unpublished labels and DLP policies stuck in **Test** are the most common findings — the controls exist but protect nothing. Weight them accordingly.

Scoring and the roadmap

- **Agree the baseline first.** A gap is the distance from **current** to **target**; if the target isn't agreed with compliance/security stakeholders, the scorecard is just opinion. Tune the default baseline before you score.
- **Rank by value ÷ effort, not by severity alone.** A "publish the label you already built" quick win often beats a high-severity item that needs a quarter of work. The roadmap's job is **sequence**.
- **Attribute every determination.** Whether a gap *must* be closed for a regulation is a compliance/records/legal call. Put the owner function on every gap row; provide the technical method and evidence, **not** the determination.
- **Make the scorecard repeatable.** Keep the discovery + validation scripts so the same measurement can be re-run after each follow-on fix — a rising score per dimension is the proof the program is working.

Piloting safely

- **Always pilot in simulation/test, never enforce.** Auto-labeling **simulation** (`TestWithoutNotifications`) labels nothing; DLP **test** blocks nothing; label pilots publish to the **pilot group only**. This engagement proves the **fix path**, it does not deploy it.
- **Preview with `-WhatIf` before `-Commit`.** The Pilot Remediation script defaults to `-WhatIf`; look at exactly what it would create before you create it.
- **Roll the pilot back cleanly.** A simulation policy labeled nothing, so removing it reverts nothing — delete it after validating, unless the follow-on engagement is starting immediately.
- **One fix, well proven.** Validate a single, high-value, low-risk fix thoroughly rather than half-piloting several. Credibility comes from one proven path, not many started ones.

Operating and handing off

- **Deliver to a decision-maker.** The roadmap is a decision artifact — present it to the accountable

compliance/security owners, not just IT, so the ranked work gets funded.

• **Enforcement is a separate, change-controlled engagement.** Turning a validated fix into an enforced

program (enable DLP, publish tenant-wide, enable auto-labeling) is follow-on work under change control — scope it explicitly, don't let it bleed into the assessment.

• **Re-assess on a cadence.** Posture drifts as the estate grows; re-run the assessment periodically (and after major M365 changes) so the baseline stays honest.

Automation & security

• **Read-only first, least privilege always.** Discovery / ScopeHealth / Validation / Report are **read-only**; Pilot Remediation defaults to `-WhatIf` and creates only a **simulation** object. Connect with the **least Graph scopes** and the least Purview role each task needs.

• **No secrets in code.** Never place credentials, tokens, or connection strings in scripts or source control; rely on interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`).

• **Don't invent capability.** Where a cmdlet or portal path isn't readable, the scripts report **"not available — verify your licensing/tenant"** rather than asserting a feature exists. Keep that discipline in the written findings too.

8. Deliverables Summary — Information Protection Assessment & Readiness (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case: baseline posture, ranked gaps, a validated fix path
1. Solution Overview	The posture-assessment method, the six dimensions assessed, pilot-in-simulation principle, what's authored vs. deferred
2. Requirements Guide	Every assessment prerequisite with Verify / Implement-if-missing (category Purview GRs referenced, not duplicated)
Global Requirements (Purview)	Category-wide Purview prerequisites, published once
3. Implementation Guide	Exact, ordered Purview portal + S&C PowerShell + Graph steps: discover (read-only), score, build gap register, roadmap, pilot ONE fix in simulation, validate, hand off — pass/fail gates + rollback
5. Testing Guide	Validation procedures with pass/fail gates

Document	Purpose
6. Troubleshooting Guide	Connectivity, licensing-gated visibility, scoring, and simulation-pilot issues by symptom → root cause
7. Best Practices	Field-proven recommendations for a defensible assessment and an executable roadmap

PowerShell scripts (Security & Compliance PowerShell / Microsoft Graph)

Script	What it does	Safety
`IPASSESS-ScopeHealth.ps1`	Scope/health helper: confirms `Connect-IPPSSession` / `Connect-MgGraph`, reports which capabilities (labels, DLP, auto-labeling, SITs) are readable, flags licensing-gated ones as "not available — verify your licensing/tenant"	Read-only
`IPASSESS-Discovery.ps1`	Posture inventory: sensitivity labels (with encryption/marking/scope), label policies (publish coverage), DLP policies + rules (Mode/Workload), auto-labeling policies (Mode), and — with `-IncludeSensitiveTypes` — sensitive information types; writes `Information Protection - Estate - Discovery` (CSV)	Read-only
`IPASSESS-PilotRemediation.ps1`	Validates ONE recommended fix in SIMULATION: creates an auto-labeling policy in `TestWithoutNotifications` mode (labels nothing) — or previews a pilot-group label publish	Default `-WhatIf`; `-Commit` creates a simulation-only object; never enforces
`IPASSESS-Validation.ps1`	Scores the estate against the readiness baseline (six dimensions) PASS/WARN/FAIL, and confirms the pilot object is in simulation mode with nothing enforced	Read-only

Script	What it does	Safety
`IPASSESS-Report.ps1`	Builds `Information Protection - Gaps - Register` (`-Section GapRegister`) and the ranked `Information Protection - Roadmap - Scorecard` (`-Section Roadmap`) with a per-dimension posture scorecard	Read-only

What the buyer receives

- This assessment guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process: **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand.**

Outcome

A measured, evidenced **information-protection posture** for Microsoft Purview — a discovery baseline, a scored gap register, a prioritized roadmap and scorecard, and **one recommended fix proven in simulation** — so leadership can see exactly where the estate stands and fund the ranked work to close the gaps. Enforcement/expansion is change-controlled follow-on; regulatory/records/legal determinations are deferred to the relevant governance functions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPASSESS-Discovery.ps1

Purpose: Read-only discovery for the Information Protection Assessment. Inventories the controls that protect the data estate today: sensitivity LABELS (with their encryption/protection and content- marking settings and scope), the LABEL POLICIES that publish them (coverage), DLP policies and rules (with Mode and Workload), AUTO-LABELING policies (with simulation-vs-enabled Mode), and — with -IncludeSensitiveTypes — the sensitive information types that define "sensitive." This is the "what protects the estate today" baseline for scoring and the roadmap. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\IPASSESS-Discovery.ps1 .\IPASSESS-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS The posture inventory written to .\IPASSESS-InformationProtection-Estate-Discovery.csv (Information Protection - Estate - Discovery) and, with -IncludeSensitiveTypes, the SIT list to .\IPASSESS-SensitiveInfoTypes-Estate-Discovery.csv. Unpublished labels, DLP stuck in Test, and classify-but-don't-encrypt labels all surface as rows for scoring. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Where a capability is unlicensed/unreadable the script records a WARN and continues rather than assuming the feature exists. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\IPASSESS-InformationProtection-Estate-Discovery.csv", [string]$SensitiveOutPath = ".\IPASSESS-SensitiveInfoTypes-Estate-Discovery.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\IPASSESS-Discovery.ps1 .\IPASSESS-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS The posture inventory written to .\IPASSESS-InformationProtection-Estate-Discovery.csv (Information Protection - Estate - Discovery) and, with -IncludeSensitiveTypes, the SIT list to .\IPASSESS-SensitiveInfoTypes-Estate-Discovery.csv. Unpublished labels, DLP stuck in Test, and classify-but-don't-encrypt labels all surface as rows for scoring. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Where a capability is unlicensed/unreadable the script records a WARN and continues rather than assuming the feature exists. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\IPASSESS-InformationProtection-Estate-Discovery.csv", [string]$SensitiveOutPath = ".\IPASSESS-SensitiveInfoTypes-Estate-Discovery.csv")

IPASSESS-PilotRemediation.ps1

Purpose: Proves the recommended remediation PATH for the top information-protection gap WITHOUT changing content or enforcing anything — the assessment's "pilot in simulation" step. With -Action AutoLabelSim it creates an AUTO-LABELING policy in SIMULATION mode (Mode = TestWithoutNotifications) that evaluates content and reports what it WOULD label and applies NO label to anything. With -Action LabelPublishPreview it prints the exact commands to publish an existing unpublished label to a small PILOT GROUP only (it does not run them unless you follow the printed guidance) so the publish path can be proven without a tenant-wide change. SAFE BY DEFAULT: prints intended actions under -WhatIf; creates the simulation policy only with -Commit; NEVER enables or enforces anything. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell

(Connect-IPPSSession). Dry-run (default, no change): .\IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim -SensitivityLabel "Confidential" -WhatIf Create the SIMULATION validation policy (labels nothing): .\IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim -SensitivityLabel "Confidential" -Commit Show the pilot-group label-publish path (prints commands only): .\IPASSESS-PilotRemediation.ps1 -Action LabelPublishPreview -SensitivityLabel "Confidential" -PilotGroup "Information Protection - Pilot Users" EXPECTED RESULTS AutoLabelSim: a policy named 'Information Protection - Pilot - Simulation' in SIMULATION mode exists (confirm in portal; Testing Test 8) and labels nothing; add a matching rule with your SIT conditions to see would-be matches. LabelPublishPreview: the exact New-LabelPolicy command scoped to the pilot group, printed for review. Nothing is created without -Commit; nothing is ever enforced. .NOTES White-label Microsoft Purview tooling. SAFE-by-default (-WhatIf); simulation/pilot-group only. No credentials stored. HIGH-IMPACT changes are piloted in simulation per the category model — this script validates the fix path and DOES NOT enforce; enforcement/expansion is a separate change-controlled follow-on. Determinations (whether to remediate) remain with compliance/records/privacy/legal. No cmdlets invented. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory = $true)] [ValidateSet("AutoLabelSim","LabelPublishPreview")] [string]$Action,

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). Dry-run (default, no change): .\IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim -SensitivityLabel "Confidential" -WhatIf Create the SIMULATION validation policy (labels nothing): .\IPASSESS-PilotRemediation.ps1 -Action AutoLabelSim -SensitivityLabel "Confidential" -Commit Show the pilot-group label-publish path (prints commands only): .\IPASSESS-PilotRemediation.ps1 -Action LabelPublishPreview -SensitivityLabel "Confidential" -PilotGroup "Information Protection - Pilot Users" EXPECTED RESULTS AutoLabelSim: a policy named 'Information Protection - Pilot - Simulation' in SIMULATION mode exists (confirm in portal; Testing Test 8) and labels nothing; add a matching rule with your SIT conditions to see would-be matches. LabelPublishPreview: the exact New-LabelPolicy command scoped to the pilot group, printed for review. Nothing is created without -Commit; nothing is ever enforced. .NOTES White-label Microsoft Purview tooling. SAFE-by-default (-WhatIf); simulation/pilot-group only. No credentials stored. HIGH-IMPACT changes are piloted in simulation per the category model — this script validates the fix path and DOES NOT enforce; enforcement/expansion is a separate change-controlled follow-on. Determinations (whether to remediate) remain with compliance/records/privacy/legal. No cmdlets invented. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory = $true)] [ValidateSet("AutoLabelSim","LabelPublishPreview")] [string]$Action,

IPASSESS-Report.ps1

Purpose: Read-only reporting for the Information Protection Assessment. Turns the scored posture (IPASSESS-Validation.ps1 output) into two deliverables: with -Section GapRegister, a gap register (one row per gap with severity, recommended action, effort, and owner-function); with -Section Roadmap, a ranked roadmap that sequences the gaps into quick wins vs. strategic work plus a one-page posture scorecard. Produces files only — changes nothing in the tenant, and makes no determination about whether a gap must be closed (that is compliance/records/legal's call). HOW TO RUN PowerShell 5.1+ or 7+ (no S&C session required — operates on the score CSV). Build the gap register: .\IPASSESS-Report.ps1 -Section GapRegister -ScoreCsv .\IPASSESS-PostureScore-Estate-Validate.csv Build the ranked roadmap + scorecard: .\IPASSESS-Report.ps1 -Section Roadmap -ScoreCsv .\IPASSESS-PostureScore-Estate-Validate.csv EXPECTED RESULTS GapRegister -> .\IPASSESS-Gaps-Register.csv (Information Protection - Gaps - Register). Roadmap -> .\IPASSESS-Roadmap-Scorecard.csv (Information Protection - Roadmap - Scorecard) with quick-win vs.

strategic sequencing and a printed per-dimension scorecard. Every gap row carries an owner-function so determinations are attributed, not decided. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. The severity/effort mapping is a starting default to refine with stakeholders; whether to act on a gap remains a compliance/records/privacy/legal determination. No cmdlets invented. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [ValidateSet("GapRegister","Roadmap")] [string]$Section,

How to run: PowerShell 5.1+ or 7+ (no S&C session required — operates on the score CSV). Build the gap register: .\IPASSESS-Report.ps1 -Section GapRegister -ScoreCsv .\IPASSESS-PostureScore-Estate-Validate.csv Build the ranked roadmap + scorecard: .\IPASSESS-Report.ps1 -Section Roadmap -ScoreCsv .\IPASSESS-PostureScore-Estate-Validate.csv EXPECTED RESULTS GapRegister -> .\IPASSESS-Gaps-Register.csv (Information Protection - Gaps - Register). Roadmap -> .\IPASSESS-Roadmap-Scorecard.csv (Information Protection - Roadmap - Scorecard) with quick-win vs. strategic sequencing and a printed per-dimension scorecard. Every gap row carries an owner-function so determinations are attributed, not decided. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. The severity/effort mapping is a starting default to refine with stakeholders; whether to act on a gap remains a compliance/records/privacy/legal determination. No cmdlets invented. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [ValidateSet("GapRegister","Roadmap")] [string]$Section,

IPASSESS-ScopeHealth.ps1

Purpose: Scope/health helper for the Information Protection Assessment. Confirms the Security & Compliance PowerShell session (Connect-IPPSSession) and, optionally, Microsoft Graph, then probes which posture capabilities are actually READABLE in this tenant — sensitivity labels, label policies, DLP, auto-labeling, and sensitive information types. Licensing-gated capabilities (auto-labeling, advanced DLP) that cannot be read are reported as "not available — verify your licensing/tenant" rather than assumed present or absent. Makes no changes. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\IPASSESS-ScopeHealth.ps1 .\IPASSESS-ScopeHealth.ps1 -CheckGraph # also verify Connect-MgGraph for the pilot group EXPECTED RESULTS A readiness line per capability (READABLE / NOT AVAILABLE) so you know which of the six posture dimensions the assessment can score before you run IPASSESS-Discovery.ps1. A "not available" line is a valid finding (licensing), not a script failure. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell / Microsoft Graph; none are invented. Feature availability is licensing- dependent — the script reports readability, it does not assert a capability exists. > [CmdletBinding()] param([switch]$CheckGraph)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\IPASSESS-ScopeHealth.ps1 .\IPASSESS-ScopeHealth.ps1 -CheckGraph # also verify Connect-MgGraph for the pilot group EXPECTED RESULTS A readiness line per capability (READABLE / NOT AVAILABLE) so you know which of the six posture dimensions the assessment can score before you run IPASSESS-Discovery.ps1. A "not available" line is a valid finding (licensing), not a script failure. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell / Microsoft Graph; none are invented. Feature availability is licensing- dependent — the script reports readability, it does not assert a capability exists. > [CmdletBinding()] param([switch]$CheckGraph)

IPASSESS-Validation.ps1

Purpose: Read-only scoring for the Information Protection Assessment. Reads the discovery CSV (or queries live) and scores the six posture dimensions against a readiness baseline: taxonomy (labels exist and enabled), publishing/coverage (labels actually published), encryption/protection (higher-tier labels encrypt), auto-labeling (present and its mode), DLP coverage/mode (in-scope workloads covered and not stuck in Test), and sensitive information types. Also verifies that any pilot object is in SIMULATION mode with nothing enforced. Emits PASS/WARN/FAIL per dimension. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). From the discovery CSV (recommended): .\IPASSESS-Validation.ps1 -DiscoveryCsv .\IPASSESS-InformationProtection-Estate-Discovery.csv ` -InScopeWorkloads "Exchange","SharePoint","OneDrive","Teams" Confirm the simulation pilot object is safe: .\IPASSESS-Validation.ps1 -CheckPilot EXPECTED RESULTS A PASS/WARN/FAIL line per dimension against the baseline, written to .\IPASSESS-PostureScore-Estate-Validate.csv. With -CheckPilot, confirmation that 'Information Protection - Pilot - Simulation' (if present) is in a TestWithoutNotifications mode and enforces nothing. Tune the baseline targets with stakeholders before treating a score as final. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. The baseline is a starting default to refine with compliance/security stakeholders; determinations (whether a gap must be closed) remain with compliance/records/privacy/legal. No cmdlets invented. > [CmdletBinding()] param([string]$DiscoveryCsv = ".\IPASSESS-InformationProtection-Estate-Discovery.csv", [string[]]$InScopeWorkloads = @("Exchange","SharePoint","OneDrive","Teams"), [string[]]$HighTierLabelPattern = @("Confidential","Restricted","Secret"), [switch]$CheckPilot, [string]$OutPath = ".\IPASSESS-PostureScore-Estate-Validate.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). From the discovery CSV (recommended): .\IPASSESS-Validation.ps1 -DiscoveryCsv .\IPASSESS-InformationProtection-Estate-Discovery.csv ` -InScopeWorkloads "Exchange","SharePoint","OneDrive","Teams" Confirm the simulation pilot object is safe: .\IPASSESS-Validation.ps1 -CheckPilot EXPECTED RESULTS A PASS/WARN/FAIL line per dimension against the baseline, written to .\IPASSESS-PostureScore-Estate-Validate.csv. With -CheckPilot, confirmation that 'Information Protection - Pilot - Simulation' (if present) is in a TestWithoutNotifications mode and enforces nothing. Tune the baseline targets with stakeholders before treating a score as final. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. The baseline is a starting default to refine with compliance/security stakeholders; determinations (whether a gap must be closed) remain with compliance/records/privacy/legal. No cmdlets invented. > [CmdletBinding()] param([string]$DiscoveryCsv = ".\IPASSESS-InformationProtection-Estate-Discovery.csv", [string[]]$InScopeWorkloads = @("Exchange","SharePoint","OneDrive","Teams"), [string[]]$HighTierLabelPattern = @("Confidential","Restricted","Secret"), [switch]$CheckPilot, [string]$OutPath = ".\IPASSESS-PostureScore-Estate-Validate.csv")